Projektdokumentation

Webshop

Modul: 223

Autoren: Cem & Indujan

Repository: github.com/Cem499/Webshop_Cem_Indujan

Abgabedatum: 15. April 2026

Inhalt

1. Einleitung 4
1.1 Projektbeschreibung 4
1.2 Zielsetzung 4
1.3 Team und Aufgabenverteilung 4
2. User Stories 5
2.1 User Story 1: Registrierung und Login 5
Akzeptanzkriterien: 5
2.2 User Story 2: Produkte durchsuchen und bestellen 5
Akzeptanzkriterienn : 5
2.3 User Story 3: Produktverwaltung als Admin 6
Akzeptanzkriterien: 6
3. Backend 7
3.1 Technologie Stack 7
3.2 Architektur 7
Verantwortlichkeiten pro Schicht 8
3.3 Datenbank-Schema 8
3.4 REST-API Endpoints 9
3.5 Transaktionen 9
4. JWT-Authentifizierung 10
4.1 Konzept 10
4.2 Login- und Request-Flow 10
4.3 Implementierung im Backend 11
4.4 Implementierung im Frontend 11
5. Frontend 12
5.1 Technologie Stack 12
5.2 Architektur 12
State Management mit AuthContext 12
API Integration 13
5.3 Wichtige Komponenten 13
6. Sicherheitskonzept 14
6.1 Überblick der Schutzmassnahmen 14
6.2 Passwort-Sicherheit 14
6.3 JWT-Sicherheit 14
6.4 Rollenbasierte Autorisierung 14
6.5 Schutz vor IDOR (Insecure Direct Object Reference) 15

7. Tests 16
7.1 Backend-Tests 16
Testprotokoll 16
7.2 Frontend-Tests 16
Testprotokoll 16
8. Git-Workflow 17
8.1 Repository und Struktur 17
8.2 Branching-Modell 17
8.3 Typischer Workflow 17
9. Setup und Inbetriebnahme 18
9.1 Voraussetzungen 18
9.2 Backend starten 18
9.3 Frontend starten 18
9.4 Tests ausführen 18
10. Fazit und Lessons Learned 19
10.1 Was gut gelaufen ist 19
10.2 Herausforderungen 19
10.3 Was wir nächstes Mal anders machen würden 19
11. Hilfsmittel 19

1. Einleitung

1.1 Projektbeschreibung

Im Modul 223 haben wir einen Webshop entwickelt. Der Webshop besteht aus einem Spring Boot Backend und einem React Frontend und nutzt MySQL als Datenbank.
Kunden können sich registrieren, einloggen, Produkte nach Kategorien durchsuchen und Bestellungen aufgeben. Administratoren können zusätzlich Produkte und Kategorien verwalten sowie alle Bestellungen einsehen. Die Authentifizierung erfolgt über JWT Tokens.

1.2 Zielsetzung

- Eine funktionsfähige Multi User Webapplikation mit klarer Trennung von Frontend und Backend entwickeln.
- Rollenbasiertes Berechtigungskonzept mit JWT umsetzen.
- Automatisierte Tests für Backend und Frontend schreiben.
- Sauberen Git Workflow mit aussagekräftigen Commits einhalten.
- Die Architektur, Sicherheitskonzepte und Technologien nachvollziehbar dokumentieren.

1.3 Team und Aufgabenverteilung

Wir haben alles zusammen gemacht, auf einem Laptop sind wir schritt für schritt zusammen durchgegangen und uns gegenseitig abgefragt was wir als nächstes machen oder was wir besser machen könnten.

2. User Stories

Die folgenden drei User Stories bilden den funktionalen Kern des Webshops ab und wurden während der gesamten Entwicklung als Referenz verwendet.

2.1 User Story 1: Registrierung und Login

Als neuer Besucher des Webshops möchte ich mich mit E-Mail und Passwort registrieren und anschliessend einloggen können, damit ich Bestellungen aufgeben und meine Bestellhistorie einsehen kann.

Akzeptanzkriterien:

- Das Registrierungsformular verlangt Benutzername, E-Mail und Passwort (mindestens 6 Zeichen).
- Bei erfolgreicher Registrierung wird der Benutzer mit die Rolle KUNDE in der Datenbank angelegt und das Passwort wird mit BCrypt gehasht gespeichert.
- Versucht ein Benutzer, sich mit einer bereits vergebenen E-Mail zu registrieren, erhält er eine Fehlermeldung.
- Beim Login mit korrekten Daten erhält der Benutzer ein JWT Token und wird zur Startseite weitergeleitet.
- Bei falschen Zugangsdaten erscheint die Fehlermeldung „Ungültige E-Mail oder Passwort" ohne Hinweis darauf, welches Feld falsch war natürlich.
- Nach dem Login bleibt der Benutzer so lange angemeldet, bis er sich ausloggt oder das Token abläuft, das Token läuft in 24h ab.

2.2 User Story 2: Produkte durchsuchen und bestellen

Als eingeloggter Kunde möchte ich Produkte in einen Warenkorb legen und eine Bestellung mit Lieferadresse aufgeben, damit ich die gewünschten Produkte beziehen kann.

Akzeptanzkriterienn :

- Alle Produkte werden mit Name, Beschreibung, Preis und verfügbarem Bestand angezeigt.
- Produkte können nach Kategorie gefiltert werden.
- Beim Hinzufügen zum Warenkorb wird geprüft, ob die gewünschte Menge im Bestand verfügbar ist.
- Beim Absenden der Bestellung wird der Bestand der bestellten Produkte transaktional reduziert. Schlägt die Bestandsprüfung für auch nur ein Produkt fehl, wird die gesamte Bestellung zurückgerollt.
- Der Gesamtbetrag wird im Backend anhand der aktuellen Produktpreise aus der Datenbank berechnet. Werte aus dem Frontend werden dabei ignoriert, damit manipulierte Preise keine Auswirkung haben.
- Nach erfolgreicher Bestellung wird der Benutzer auf die Bestellübersicht weitergeleitet und sieht seine neue Bestellung mit Status OFFEN.

2.3 User Story 3: Produktverwaltung als Admin

Als Administrator möchte ich Produkte und Kategorien anlegen, bearbeiten und löschen können, damit ich das Sortiment des Webshops aktuell halten kann.

Akzeptanzkriterien:

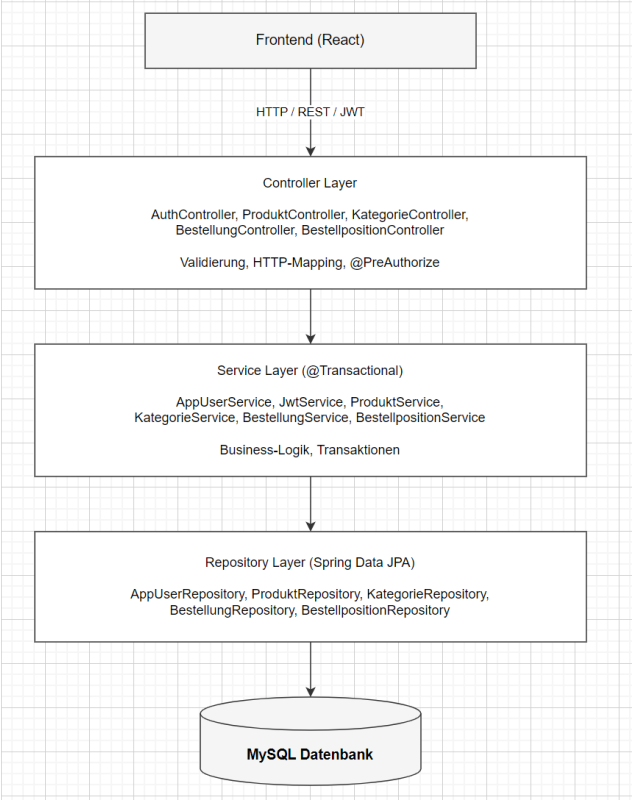
- Nur Benutzer mit der Rolle ADMIN sehen die Menüpunkte „Produkte verwalten" und „Kategorien verwalten".
- Versucht ein Benutzer mit Rolle KUNDE einen Admin Endpoint direkt aufzurufen, antwortet das Backend mit HTTP 403 Forbidden.
- Beim Anlegen eines Produkts müssen Name, Preis (grösser als 0), Bestand (nicht negativ) und eine existierende Kategorie angegeben werden.
- Ungültige Eingaben werden durch Bean Validation abgefangen und als Validierungsfehler mit HTTP 400 zurückgegeben.
- Beim Löschen einer Kategorie, die noch Produkte enthält, wird die Operation verhindert, um Dateninkonsistenzen zu vermeiden.
- Alle Änderungen am Sortiment sind sofort für Kunden sichtbar, ohne dass diese die Seite neu laden müssen (nach dem nächsten Navigations Wechsel).

3. Backend

3.1 Technologie Stack

Technologie	Version	Zweck
Java	17	Programmiersprache
Spring Boot	3.3.2	Framework für REST-API und Dependency Injection
Spring Security	6.x	Authentifizierung und Autorisierung
JJWT	0.12.3	JWT Token erzeugen, parsen und validieren
Spring Data JPA	3.x	ORM und Repository Pattern
Hibernate	6.x	JPA-Implementierung
MySQL	8.x	Relationale Datenbank
BCrypt	via Spring Security	Passwort-Hashing
springdoc-openapi	2.5.0	Swagger UI für API-Dokumentation
JUnit 5 + Mockito	5.x / 5.x	Unit- und Integrationstests
Maven	3.x	Build- und Dependency-Management

3.2 Architektur



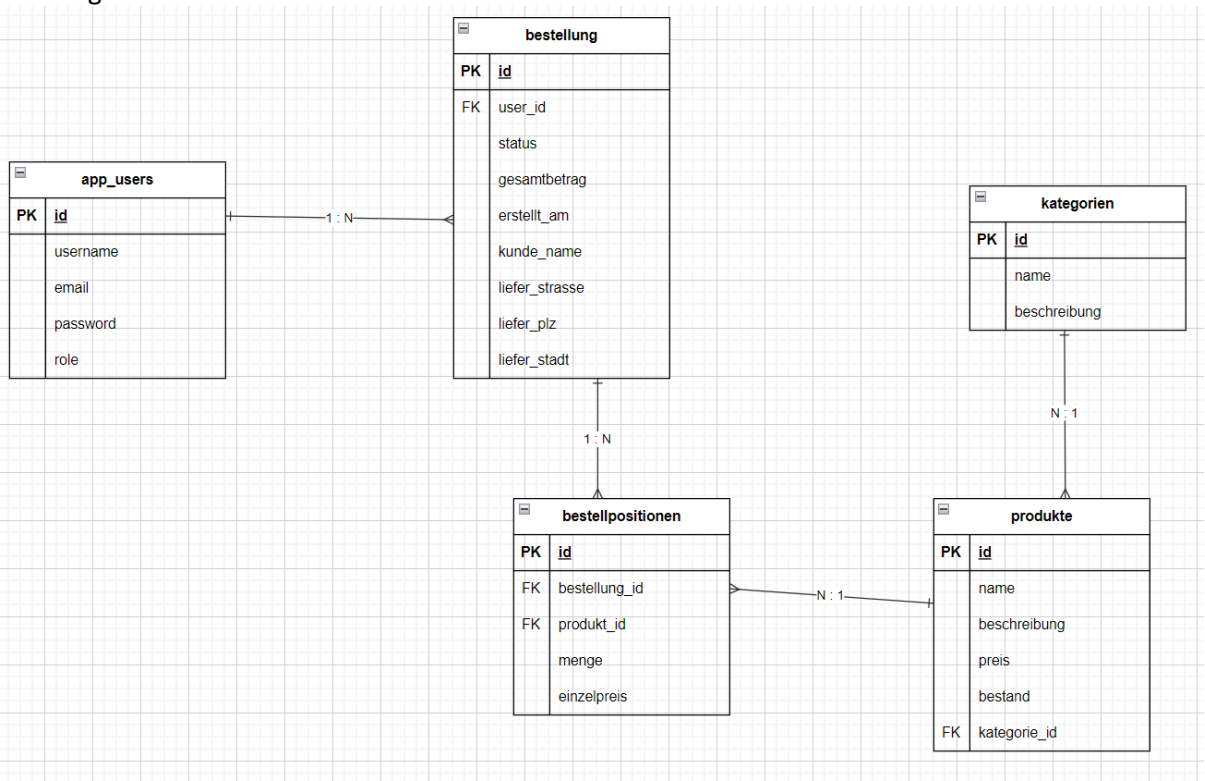
Das Backend folgt einer klassischen Schichtenarchitektur (Layered Architecture). Jede Schicht hat eine klar abgegrenzte Verantwortung und kennt nur die direkt darunterliegende Schicht. Diese Trennung erleichtert Testing, Wartung und spätere Erweiterungen.

Verantwortlichkeiten pro Schicht

- **Controller Layer:** Nimmt HTTP-Requests entgegen, validiert Eingaben mit Bean Validation, prüft Berechtigungen mit @PreAuthorize und gibt passende HTTP-Statuscodes zurück. Enthält selbst keine Businesslogik.
- **Service Layer:** Enthält die eigentliche Geschäftslogik. Services sind mit @Transactional annotiert, sodass zusammenhängende Datenbankoperationen in einer Transaktion laufen. Schlägt eine Operation fehl, werden alle Änderungen zurückgerollt.
- **Repository Layer:** Zugriff auf die Datenbank über Spring Data JPA. Repositories sind Interfaces, Spring generiert die Implementierung mit passenden SQL Queries zur Laufzeit.
- **Datenbank:** MySQL speichert alle persistenten Daten. Das Schema wird von Hibernate beim Startup automatisch erstellt bzw. migriert (ddl-auto=update).

3.3 Datenbank-Schema

Das Datenmodell umfasst fünf Tabellen. Die zentrale Tabelle app_users speichert alle Benutzer mit ihrer Rolle. Produkte sind über eine N:1-Beziehung einer Kategorie zugeordnet. Eine Bestellung gehört einen Benutzer und enthält über Bestellpositionen mehrere Produkte mit Mengen und Preisangaben.



3.4 REST-API Endpoints

Die folgenden Endpoints bilden die öffentliche Schnittstelle des Backends. Die Spalte Rolle beschreibt, welche Autorisierung der aufrufende Benutzer mindestens benötigt.

Methode	Pfad	Rolle	Zweck
POST	/api/auth/register	Public	Neuen Kunden registrieren
POST	/api/auth/login	Public	Login, gibt JWT Token zurück
GET	/api/auth/me	Authenticated	Eigene Benutzerdaten abrufen
GET	/api/produkte	Public	Alle Produkte auflisten
GET	/api/produkte/{id}	Public	Einzelnes Produkt abrufen
POST	/api/produkte	ADMIN	Produkt anlegen
PUT	/api/produkte/{id}	ADMIN	Produkt bearbeiten
DELETE	/api/produkte/{id}	ADMIN	Produkt löschen
GET	/api/kategorien	Public	Kategorien auflisten
POST	/api/kategorien	ADMIN	Kategorie anlegen
GET	/api/bestellungen	Authenticated	Eigene Bestellungen (Admin: alle)
POST	/api/bestellungen	Authenticated	Neue Bestellung aufgeben
POST	/api/bestellungen/{id}/bezahlen	Authenticated	Bestellung als bezahlt markieren
POST	/api/bestellungen/{id}/stornieren	Authenticated	Bestellung stornieren

3.5 Transaktionen

Alle Services im Backend sind mit @Transactional annotiert. Damit stellt Spring sicher, dass zusammengehörende Datenbankoperationen entweder vollständig oder gar nicht ausgeführt werden. Besonders kritisch ist dies im BestellungService.
Beim Aufgeben einer Bestellung werden mehrere Datensätze gleichzeitig verändert. Es wird ein Bestellungs Datensatz erstellt, mehrere Bestellpositionen werden angelegt und der Bestand der jeweiligen Produkte wird reduziert. Würde eine dieser Operationen fehlschlagen, zum Beispiel weil der Bestand eines Produkts nicht ausreicht und eine IllegalArgumentException geworfen wird, rollt Spring automatisch alle bereits durchgeführten Änderungen zurück. Dadurch ist garantiert, dass die Datenbank nie in einem inkonsistenten Zustand verbleibt.
Noch Beispiel ist das Bezahlen einer Bestellung: Hier wird der Bestand aller Positionen reduziert und der Status der Bestellung auf BEZAHLT gesetzt. Auch diese Operation läuft in einer einzigen Transaktion.

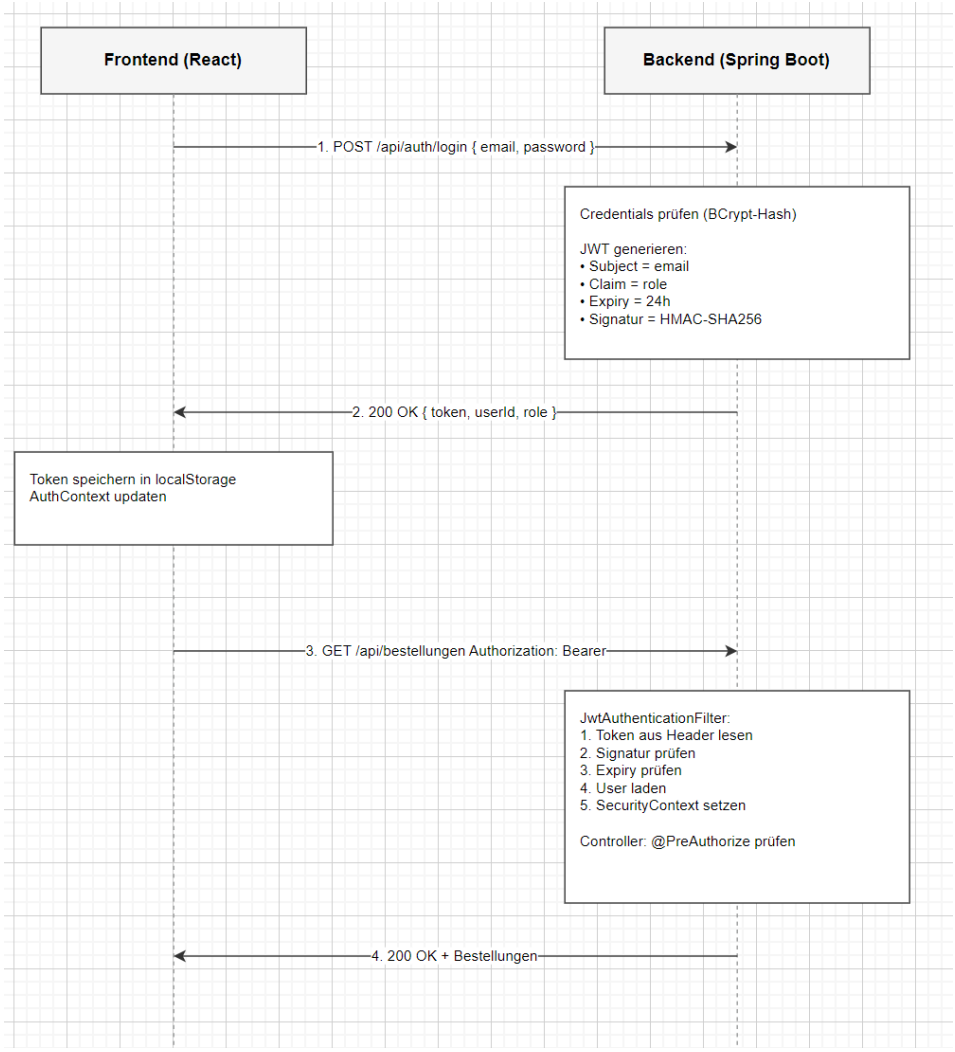
4. JWT-Authentifizierung

4.1 Konzept

Wenn du dich einloggst, erhältst du ein Token. Dieses Token wird automatisch bei jeder Anfrage mit. Der Server prüft es und weiss dadurch, wer du bist. Er muss sich nichts speichern, da alle nötigen Informationen im Token enthalten sind.

4.2 Login- und Request-Flow

Beim Login schickt der Nutzer E-Mail und Passwort an den Server. Der Server vergleicht das Passwort mit dem gespeicherten Hash in der Datenbank. Stimmt alles, erstellt er ein Token mit der E-Mail, der Rolle und einer Gültigkeit von 24 Stunden. Das Token wird signiert damit es nicht gefälscht werden kann.
Bei jeder weiteren Anfrage prüft der Server das Token sofort ab. Er schaut, ob die Signatur stimmt und ob das Token noch gültig ist, lädt dann den Nutzer aaus der Datenbank und weiss dadurch welche Rechte er hat. Erst danach darf die eigentliche Funktion ausgeführt werden.



4.3 Implementierung im Backend

Der JwtService ist zuständig für alles rund ums Token: erstellen, lesen und prüfen. Er arbeitet unabhängig vom Rest des Sicherheitssystems.
Der JwtAuthenticationFilter läuft bei jeder Anfrage genau einmal und wird früh in die Verarbeitungskette eingehängt, noch bevor der normale Login Filter greift.
Die Berechtigungen werden direkt an den Controller-Methoden mit @PreAuthorize festgelegt. Dadurch braucht man keine manuellen Rollen Checks im Code und sieht sofor wer was darf.

4.4 Implementierung im Frontend

Im Frontend verwaltet der AuthContext das Token und den eingeloggten Nutzer. Das Token wird im localStorage gespeichert, damit man nach einem Neuladen der Seite eingeloggt bleibt.
Ein Interceptor hängt das Token automatisch an jede Anfrage an, ohne dass man es manuell machen muss.
Ein zweiter Interceptor überwacht die Antworten des Servers. Kommt ein 401 zurück, also "nicht autorisiert", wird das Token gelöscht und der Nutzer zur Login Seite weitergeleitet. So werden abgelaufene Tokens automatisch erkannt und behandelt.

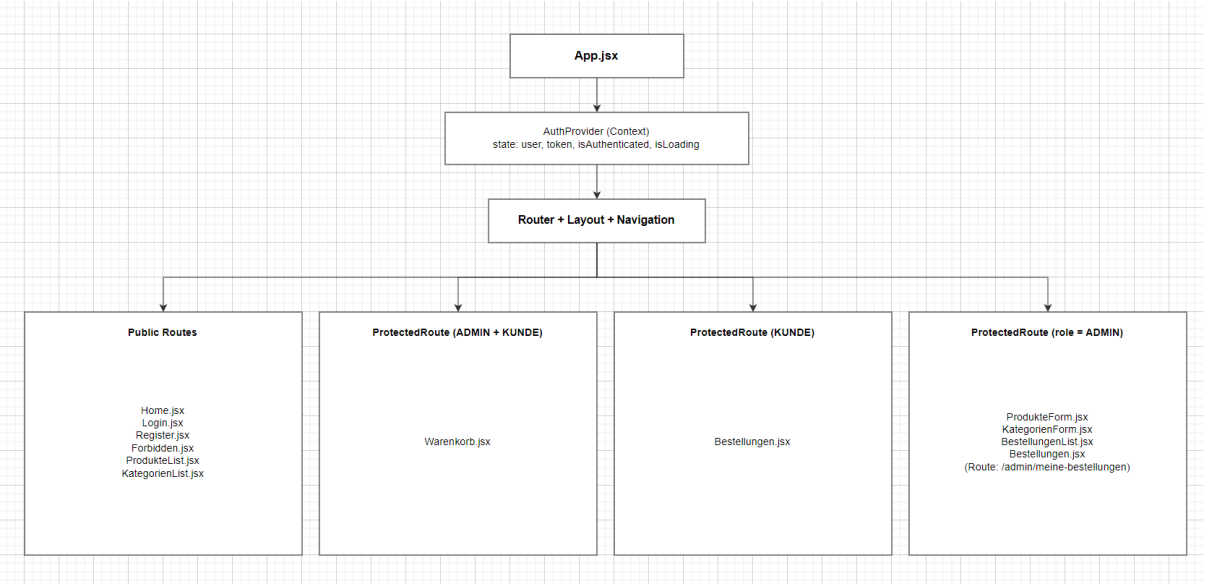
5. Frontend

5.1 Technologie Stack

Technologie	Version	Zweck
React	19.2	UI-Library
Vite	7.x	Build-Tool und Dev-Server
React Router	7.x	Client-Side Routing
Axios	1.14	HTTP-Client für API-Calls
Context API	React-native	Globales State-Management (Auth)
Jest	29.x	Test-Runner
React Testing Library	16.x	Komponenten-Tests

5.2 Architektur

Das Frontend ist in drei Bereiche aufgeteilt. In /pages liegen die ganzen Seiten, also alles was einer Route entspricht. In /components liegen kleinere Bausteine die man mehrfach verwenden kann. In /services liegt die gesamte Kommunikation mit dem Backend.

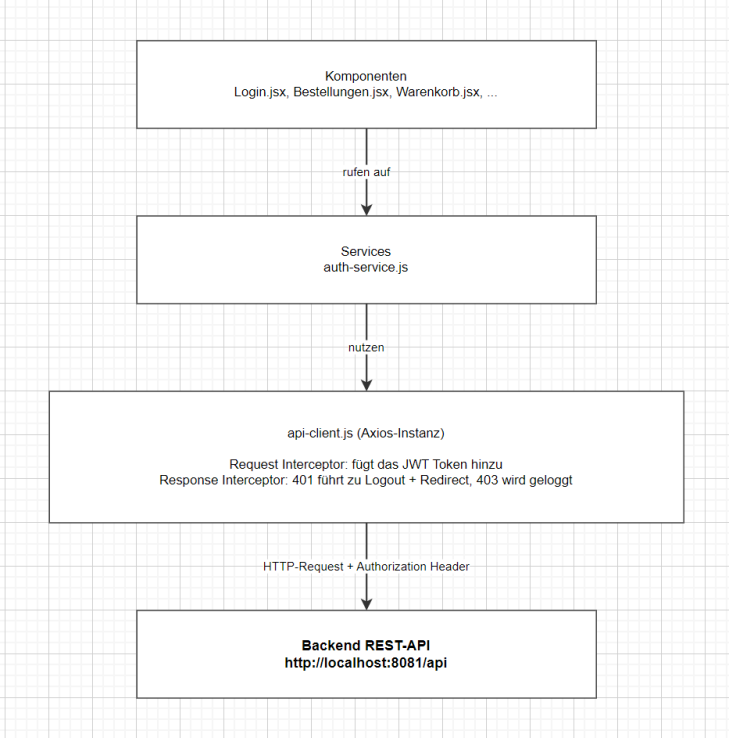


State Management mit AuthContext

Der AuthContext stellt den eingeloggten Nutzer und das Token für die gesamte App bereit. Jede Komponente kann über useAuth() darauf zugreifen und login() oder logout() aufrufen. Beim Login werden Nutzer und Token gespeichert, beim Logout wieder entfernt.

API Integration

Der Aufbau ist bewusst dreistufig. Komponenten rufen Service Funktionen auf, die Services nutzen eine zentrale Axios Instanz, und diese hängt das Token automatisch an jeden Request. Keine Komponente muss sich um AuthHeader oder URLs kümmern.



5.3 Wichtige Komponenten

- **ProtectedRoute:** Wrapper Komponente, die Kinder nur rendert, wenn der Benutzer eingeloggt ist und optional eine bestimmte Rolle hat. Sonst Weiterleitung auf /login oder /forbidden.
- **AuthContext:** Stellt die globale Auth Information bereit und synchronisiert den Zustand mit localStorage.
- **BestellungenList:** Zeigt Bestellungen des eingeloggten Benutzers (bzw. aller Benutzer, wenn Admin) mit Buttons zum Bezahlen und Stornieren.
- **ProdukteForm / KategorienForm:** Admin-only Formulare zum Anlegen und Bearbeiten von Produkten und Kategorien.

6. Sicherheitskonzept

6.1 Überblick der Schutzmassnahmen

Bedrohung	Gegenmassnahme
Unverschlüsseltes Passwort	BCrypt-Hashing vor Speicherung in DB
Session-Hijacking	Stateless JWT, keine serverseitigen Sessions
Token-Manipulation	HMAC-SHA256 Signatur mit Server-Secret
Abgelaufene Tokens	Expiration-Claim, Validierung bei jedem Request
Unbefugter Zugriff auf Admin-Funktionen	@PreAuthorize("hasRole('ADMIN')")
Zugriff auf fremde Bestellungen (IDOR)	Ownership-Check im BestellungController
SQL-Injection	JPA / Prepared Statements (keine String-Konkatenation)
CSRF	Nicht anwendbar bei stateless JWT (kein Session-Cookie)
Ungültige Eingaben	Bean Validation (@NotBlank, @Email, @Min, @DecimalMin)

6.2 Passwort-Sicherheit

Passwörter werden niemals im Klartext gespeichert. Beim Registrieren hasht der AppUserService das Passwort mit dem BCryptPasswordEncoder. BCrypt verwendet pro Passwort einen zufälligen Salt und ist bewusst langsam, um Brute-Force-Angriffe zu erschweren. Selbst bei einem Datenleck wären die Passwörter nicht direkt nutzbar.

6.3 JWT-Sicherheit

- Das JWT wird mit HMACSHA256 signiert. Ein manipuliertes Token fällt bei der Signaturprüfung sofort auf und wird vom Backend abgelehnt.
- Der Secret-Key hat 256 Bit und liegt aktuell in application.properties. In einem produktiven Einsatz würde er als Umgebungsvariable gesetzt und nicht im Repository eingecheckt.
- Tokens sind 24 Stunden gültig. Nach Ablauf muss der Benutzer sich erneut einloggen.
- Das Token enthält keine sensiblen Daten (nur E-Mail und Rolle), da JWT-Payloads Base64-kodiert und nicht verschlüsselt sind und somit von jedem gelesen werden können.

6.4 Rollenbasierte Autorisierung

Die Autorisierung erfolgt zweistufig. Einerseits schränkt die SecurityConfig global ein, welche Pfade überhaupt ohne Token erreichbar sind. Andererseits prüft @PreAuthorize pro Methode die erforderliche Rolle. Dadurch ist auch sichergestellt, dass das Backend ohne das Frontend geschützt bleibt, selbst wenn ein Angreifer Endpoints direkt via Postman oder curl aufruft.

6.5 Schutz vor IDOR (Insecure Direct Object Reference)

Im BestellungController wird nicht nur geprüft ob der Nutzer eingeloggt ist, sondern auch ob die Bestellung ihm gehört oder er ein Admin ist. Ohne diese Prüfung könnte ein Kunde einfach die ID in der URL ändern und fremde Bestellungen einsehen, obwohl er authentifiziert ist. Dieser Fehler war in einer frühen Version vorhanden und wurde im Rahmen der Sicherheitsanalyse behoben.

7. Tests

7.1 Backend-Tests

Das Backend enthält fünf Testklassen, die sowohl die Service Schicht als auch die Controller Schicht abdecken. Die Tests verwenden JUnit 5 für das Test Framework und Mockito zum Mocken der Abhängigkeiten.

Testklasse	Typ	Was wird getestet
JwtServiceTest	Unit	Token-Generierung und -Validierung
AppUserServiceTest	Unit	Registrierung, Duplicate-Erkennung, Login
KategorieServiceTest	Unit	CRUD-Operationen auf Kategorien
AuthControllerTest	Integration	Register-/Login-Endpoints inkl. HTTP-Status
ProduktTest	Unit	Bestandslogik (reduzieren, erhöhen, Validierung)

Testprotokoll

Die Tests werden mit mvn test ausgeführt. Alle Testklassen laufen erfolgreich durch, die Reports liegen unter target/surefire-reports/ und werden bei jedem Build automatisch erzeugt.

7.2 Frontend-Tests

Das Frontend verwendet Jest als Test-Runner und React Testing Library für komponentenbasierte Tests. Getestet werden die sicherheitskritischen Bestandteile, also der AuthContext und die ProtectedRoute-Komponente.

Testdatei	Was wird getestet
AuthContext.test.jsx	Login setzt Token und User, Logout räumt State und localStorage auf
ProtectedRoute.test.jsx	Weiterleitung auf /login wenn nicht authentifiziert, Rendern der Kinder bei eingeloggtem Benutzer

Testprotokoll

Die Frontend-Tests werden mit npm test ausgeführt. Jest gibt im Terminal einen zusammenfassenden Report mit Anzahl bestandener und fehlgeschlagener Tests aus.

8. Git-Workflow

8.1 Repository und Struktur

Das Projekt liegt in einem Repository auf GitHub unter Cem499/Webshop_Cem_Indujan. Backend und Frontend teilen sich ein gemeinsames Repository, liegen aber in separaten Unterordnern (Webshop-Backend/ und Frontend-webshop/).

8.2 Branching-Modell

Wir haben das gesamte Projekt gemeinsam an einem Laptop entwickelt, also im Pair Programming. Da immer nur eine Person gleichzeitig committet hat und wir uns direkt absprechen konnten, haben wir bewusst auf Feature-Branches verzichtet und direkt auf dem main Branch gearbeitet. Merge Konflikte konnten so gar nicht erst entstehen. Stattdessen haben wir Wert auf saubere, aussagekräftige Commits gelegt. Insgesamt sind rund 38 Commits entstanden, die den gesamten Entwicklungsprozess nachvollziehbar machen.

8.3 Typischer Workflow

Unser Ablauf war einfach: gemeinsam am Code arbeiten, regelmässig kleine Commits mit klaren Messages erstellen und am Ende der Session pushen. Die Commit Messages wurden bewusst ohne Umlaute geschrieben, um Kompatibilitätsprobleme mit älteren Tools und Terminals zu vermeiden.

9. Setup und Inbetriebnahme

9.1 Voraussetzungen

- Java 17 JDK
- Maven 3.8 oder neuer
- MySQL 8.x mit Datenbank LB_shop_modul_295 und Benutzer manager / managerpw
- Node.js 20 oder neuer inklusive npm

9.2 Backend starten

Im Ordner Webshop-Backend/Webshop wird mit mvn clean install gebaut und anschliessend mit mvn spring-boot:run gestartet. Das Backend läuft danach auf Port 8081. Die Swagger UI ist unter /swagger-ui.html erreichbar und zeigt alle REST-Endpoints inklusive Beispiel-Requests.

9.3 Frontend starten

Im Ordner Frontend-webshop werden zuerst die Abhängigkeiten mit npm install installiert. Anschliessend startet npm run dev den Vite Dev Server auf Port 5173. Das Frontend verbindet sich automatisch mit dem Backend auf Port 8081.

9.4 Tests ausführen

Backend-Tests werden im Backend Ordner mit mvn test ausgeführt, Frontend-Tests im Frontend Ordner mit npm test. Beide Kommandos geben am Ende eine Zusammenfassung mit Anzahl bestandener Tests aus.

10. Fazit und Lessons Learned

10.1 Was gut gelaufen ist

- Die Schichtenarchitektur hat sich bewährt. Änderungen liessen sich lokal auf eine Schicht begrenzen, ohne andere Teile zu beeinflussen.
- Das Pair Programming an einem Laptop hat uns geholfen, Entscheidungen direkt gemeinsam zu treffen und Fehler früh zu erkennen, weil immer vier Augen auf den Code geschaut haben.
- @Transactional hat uns bei der Bestelllogik einen zuverlässigen Rollback Mechanismus gegeben, ohne dass wir uns selbst um die Transaktions Verwaltung kümmern mussten.

10.2 Herausforderungen

- Die Migration der Package Struktur von com.wiss.quizbackend nach ch.wiss.webshop führte zu einem ClassNotFoundException auf die Role-Klasse, verursacht durch stale Imports in einzelnen Dateien.
- Ein initialer IDOR-Bug im BestellungController (jeder authentifizierte User konnte fremde Bestellungen einsehen) fiel erst bei manuellem Testen auf und wurde nachträglich behoben.
- Der AuthController gab ursprünglich bei jedem Fehler einen 401 zurück, was das Debugging erschwert hat. Wir haben die Fehlerbehandlung gezielt verbessert und geben nun passendere Statuscodes zurück.

10.3 Was wir nächstes Mal anders machen würden

- Früher mit Integrationstests starten. Einige der oben genannten Bugs hätten Tests direkt gefunden.
- DTOs konsequenter einsetzen, anstatt Entitäten direkt zurückzugeben. Das würde die Antworten schlanker und die API stabiler machen.
- Refresh Tokens implementieren, damit Benutzer nicht nach 24 Stunden zwingend ausgeloggt werden.

11. Hilfsmittel

- **Offizielle Dokumentationen:** Spring Boot docs, Spring Security Reference, React Docs, JJWT Dokumentation.
- **Stack Overflow:** für konkrete Fehlermeldungen und Stacktraces
- **Claude:** zum Debuggen, für Erklärungen zu Spring-Konzepten und Code-Reviews
- **Unterrichtsmaterial WISS:** Modul 223, insbesondere das Quiz-Backend als Referenz für die JWT-Implementierung
- **Teamkollege:** während des Pair Programming haben wir uns laufend erklärt, was wir gerade tun. Das hat bei Problemen oft schon zur Lösung geführt.